RNAverse / README.md

Chiyasa update readme f4bb672 · 17 minutes ago History

Preview Code Blame Raw

RNAverse

This is a PyTorch implementation of our study:

RNAverse: RNA versatile embedding representation space for engineering

Table of Contents

- Installation
- Download Model Weights and Benchmark Datasets
- Inference: Extract Embeddings
  - Option 1: Python API
  - Option 2: Command Line
- Downstream Task Training

- Zero-Shot In-Painting
- Fine-Tuned In-Painting
- Iterative Optimization

🔧 Installation

Please run the following command to check your CUDA version before installing RNAverse:

```
nvidia-smi
```

or

```
nvcc --version
```

❗ Note: To install **Torch** versions compatible with your CUDA setup, please refer to the following URLs:

- Torch: https://pytorch.org/get-started/previous-versions/

The installation commands below reflect the development environment of RNAverse (PyTorch 2.1.0 + CUDA 12.1). If your setup requires a different version (e.g., an older CUDA or a different PyTorch release), please modify the PyTorch installation as needed, but compatibility is not guaranteed.

```
conda create -n rnaverse python=3.10.12
conda activate rnaverse
pip install numpy==1.26.1
pip install torch==2.1.0 torchvision==0.16.0 torchaudio==2.1.0 --index-url https://download.pytorch.org/whl/cu121
pip install rnaverse-0.1.0-py3-none-any.whl
```

⬇️ Download Model Weights and Benchmark Datasets

We have uploaded the model weights and benchmark datasets of RNAverse to Zenodo. Please download them from the links below, place them in the respective directories, and unzip them:

- **Model weights** (RNAverse, RNAverse-aptamer, RNAverse-UTR, RNAverse-SHAPE) can be downloaded from Zenodo and should be placed into the `./model_weights/` directory.
- **Benchmark datasets** can be downloaded from Zenodo and should be placed into the `./data/` directory.

Alternatively, you can use the following commands:

```
git clone https://github.com/Chiyasa/RNAverse.git

cd RNAverse
# Download model weights (RNAverse, RNAverse-aptamer, RNAverse-UTR, and RNAverse-SHAPE)
# into the ./model_weights/ directory
cd ./model_weights
wget https://zenodo.org/records/20527121/files/rnaverse.zip
unzip rnaverse.zip

wget https://zenodo.org/records/20527121/files/rnaverse_aptamer.zip
unzip rnaverse_aptamer.zip

wget https://zenodo.org/records/20527121/files/rnaverse_utr.zip
unzip rnaverse_utr.zip

wget https://zenodo.org/records/20527121/files/rnaverse_shape.zip
unzip rnaverse_shape.zip

# Download benchmark datasets into the ./data/ directory
cd ../data
wget https://zenodo.org/records/20526662/files/benchmarks.zip
unzip benchmarks.zip
```

🧠 Inference: Extract Embeddings

Option 1: Python API

```python
import torch
import numpy as np
from rnaverse import RNAverseForMaskedLM, RNAverseTokenizer

# 1. Load model & tokenizer
model_path = "./model_weights/rnaverse" # path to RNAverse
tokenizer = RNAverseTokenizer.from_pretrained(model_path)
model = RNAverseForMaskedLM.from_pretrained(model_path)
model.eval()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# 2. Prepare input data
# Each item contains:
# - 'seq' (required): RNA sequence string (e.g., "AUGC").
# - 'struct_score_list' (optional): list of floats (length = len(seq)), normalized structural scores (0-1) or null value -1
# - 'probe_method' (optional): experimental probing technology (e.g., 'icSHAPE', 'DMS-seq'). Default: 'UNK'.
# - 'rfam_id' (optional): Rfam family ID (e.g., 'RF00001'). If provided, it overrides 'func_list'.
# - 'func_list' (optional): list of functional annotation strings (e.g., Rfam IDs). Default: ['UNK'].
# - 'mask_pos_list' (optional): 0-based indices in the sequence to be replaced with the MASK token.
#   IMPORTANT: masking only affects the sequence tokens (input_ids). Structural scores (struct_score_list)
#   and functional annotations (func_list) remain unchanged at those positions.

input_dict_list = [
    # We demonstrate the different types of inputs accepted by the model.
    # 1) Only sequence -- all other fields take default values
    {
        'seq': 'UGAA',
    },
    # 2) Explicitly writing the defaults for the above (same effect)
```

```
    {
        'seq': 'UGAA',
        'struct_score_list': [-1, -1, -1, -1],    # null values for all positions
        'probe_method': 'UNK',
        'func_list': ['UNK'],
        'mask_pos_list': [],
    },
    # 3) Sequence with structural scores (some missing, some valid) and a specific probing method
    {
        'seq': 'ACGU',
        'struct_score_list': [0.42, -1, 0.87, 0.12],   # -1 indicates missing data
        'probe_method': 'icSHAPE',                     # overrides default 'UNK'
        # func_list and mask_pos_list use defaults
    },
    # 4) Sequence with functional annotations via func_list
    {
        'seq': 'GGCCU',
        'func_list': ["riboswitch","Cis-reg","RF00167","GO:0009113"],
        # struct_score_list and probe_method default
    },
    # 5) Sequence with Rfam ID -- automatically looks up the corresponding func_list from /path/to/RNAverse/rfam_map.json
    {
        'seq': 'GGCCU',
        'rfam_id': 'RF00167',        # overrides any explicit 'func_list' if present
        # If rfam_id is not found in the map, func_list becomes ['UNK'] and a warning is logged.
    },
    # 6) Sequence with masked positions (e.g., for MLM pretraining)
    # Only the sequence tokens at indices 2 and 5 become MASK; structural scores and func_list are untouched.
    {
        'seq': 'AUGCGCAU',
        'mask_pos_list': [2, 5],      # positions 0-based
        # other fields default
    },
    # 7) All fields provided explicitly (full specification)
    {
        'seq': 'UCCGAUUGC',
```

```
        'struct_score_list': [0.13, 0.51, 0.32, 0.94, -1, 0.27, 0.88, 0.477, 0.631],
        'probe_method': 'icSHAPE',
        'func_list': ["riboswitch","Cis-reg","RF00167","GO:0009113"],
        'mask_pos_list': [4],         # mask the 5th nucleotide (index 4) -- only the sequence token
    },
]

# 3. Tokenize
inputs = tokenizer(input_dict_list, padding=True, return_tensors="pt")
inputs = {k: v.to(device) for k, v in inputs.items()}

# 4. Run inference
with torch.no_grad():
    inputs["output_hidden_states"] = True
    outputs = model(**inputs)

# 5. Extract embeddings (last hidden state, skip CLS token). RNAverse does not use a SEP token at the end
hidden_states = outputs.hidden_states[-1][:, 1:, :]  # (batch, seq_len, hidden_size)

# Option A: per-position embeddings
embed_per_pos = hidden_states.cpu().numpy()            # shape: (batch, seq_len, hidden_size)

# Option B: average-pooled embedding per sequence
embed_ave = hidden_states.mean(dim=1).cpu().numpy()    # shape: (batch, hidden_size)

# Option C: CLS embedding per sequence
embed_cls = outputs.hidden_states[-1][:, 0, :]

print(f"Per-position embedding shape: {embed_per_pos.shape}")
print(f"Average-pooled embedding shape: {embed_ave.shape}")
print(f"CLS embedding shape: {embed_cls.shape}")
```

Option 2: Command Line

Prepare an input file following the same format as ./examples/input_demo.txt. For demonstration, the example below uses ./examples/input_demo.txt directly. Then run:

```
# Ensure that your current working directory is the 'RNAverse' folder.
python ./examples/infer_embed.py \
    --model_name_or_path ./model_weights/rnaverse \
    --input_txt ./examples/input_demo.txt \
    --output_dir ./output/embed \
    --output_embed_format ave \
    --batch_size 64
```

❗ Embeddings will be saved as `.pkl` files in the output directory.

🔬 Downstream Task Training

Taking the Structure Score Imputation task on the HEK293 dataset as an example:

```
# Ensure that your current working directory is the 'RNAverse' folder.
python3 ./benchmarks/multimodal/train_ssi.py \
    --model_name_or_path ./model_weights/rnaverse \
    --data_path ./data/benchmarks/multimodal/SSI \
    --data_train_path 293_train.txt --data_val_path 293_val.txt --data_test_path 293_test.txt \
    --run_name rnaverse_ssi_293_seed42 \
    --per_device_train_batch_size 32 \
    --per_device_eval_batch_size 32 \
    --gradient_accumulation_steps 1 \
    --learning_rate 3e-5 \
    --num_train_epochs 30 \
    --fp16 \
    --save_steps 400 \
    --output_dir ./output/downstream/SSI_293_seed42 \
    --save_strategy steps \
```

```
    --eval_strategy steps \
    --eval_steps 200 \
    --warmup_steps 50 \
    --logging_steps 200 \
    --overwrite_output_dir True \
    --log_level info \
    --seed 42 \
    --model_type rnaverse
```

🚀 Zero-Shot In-Painting

RNAverse can directly design RNA functional classes that were included in the multimodal alignment stage using the zero-shot in-painting strategy. Below is an example for the design of RNAverse-gACA19:

> ❗ **Note:** In the initial FASTA sequence provided to `--init_seq` in `./generation/in_painting.py`, the character `N` denotes positions where new nucleotides will be generated (i.e., masked positions). All other characters (A, U, C, G) remain fixed.

```
# Ensure that your current working directory is the 'RNAverse' folder.
python3 ./generation/in_painting.py \
    --init_seq ./data/design/RNAverse_gACA19_init.fa \
    --model_name_or_path ./model_weights/rnaverse \
    --rfam_id RF00413 \
    --random_seed 42 \
    --seq_num 5000 \
    --max_token_per_iter 5 \
    --output_fa ./output/design/RNAverse_gACA19.fa
```

🎨 Fine-Tuned In-Painting

For RNA functional classes that are absent from the alignment corpus, RNAverse can be fine-tuned with the MLM objective using sequences from the target RNA class. Below is an example using IGFBP3 aptamer SELEX sequences.

> ❗ **Note:** The `func_vocab.json` and `rfam_map.json` files should be edited first to include the novel functional class, as shown in `./model_weights/rnaverse-aptamer/func_vocab.json` and `./model_weights/rnaverse-aptamer/rfam_map.json`.

```
# Ensure that your current working directory is the 'RNAverse' folder.
python3 ./examples/ft_mlm.py \
    --batch_size 32 \
    --lr 5e-6 \
    --pad_len 1025 \
    --random_seed 37 \
    --model_path ./model_weights/rnaverse \
    --config_path ./model_weights/rnaverse-aptamer \
    --tokenizer_path ./model_weights/rnaverse-aptamer \
    --save_model_dir ./output/design/ft_mlm/ \
    --max_epoch 200 \
    --tol_epoch 8 \
    --whether_ft 1 \
    --train_txt ./data/design/datasetA_IGFBP3_P6_train_rnaverse.txt \
    --test_txt ./data/design/datasetA_IGFBP3_P6_test_rnaverse.txt
```

We provide the trained model weights of RNAverse-aptamer, which can be directly used for in-painting generation of the IGFBP3 aptamer:

```
# Ensure that your current working directory is the 'RNAverse' folder.
python3 ./generation/in_painting.py \
    --init_seq ./data/design/RNAverse_aptamer_init.fa \
    --model_name_or_path ./model_weights/rnaverse-aptamer \
    --rfam_id IGFBP3_aptamer \
    --random_seed 42 \
    --seq_num 5000 \
    --max_token_per_iter 1 \
    --output_fa ./output/design/RNAverse_aptamer.fa
```

Iterative Optimization

We provide the model weights of RNAverse-UTR and RNAverse-SHAPE, which can be directly used for iterative optimization to design 5′ UTR sequences:

```
# Ensure that your current working directory is the 'RNAverse' folder.
python3 ./generation/iterative_optimization.py \
    --out_dir ./output/design/iter_opt_utr5/ \
    --cds ATGGTGAGCAAGGGCGAGGAGCTGTTCACCGGGGTGGTGCCCATCCTGGT \
    --design_length 150 \
    --nbr_sequences 500 \
    --evolved_iter 500 \
    --nbr_bases_to_mutate 6 \
    --rnaverse_shape_model_path ./model_weights/rnaverse-shape \
    --rnaverse_utr_model_file ./model_weights/rnaverse-utr/best_model.pt \
    --bs 32 \
    --random_seed 42
```